Finask: An All-in-One University Selection Guidance Platform

1. Introduction

The transition from high school to higher education is a critical juncture for Ethiopian youth. Following the Ethiopian Secondary School Leaving Certificate Examination (ESSLCE), Grade 12 students are required to select their preferred universities and fields of study for Ministry of Education (MoE) placement. However, they are forced to make these life-altering decisions in an information vacuum.

Finask (derived from the Ge'ez words "Fnote" meaning path, and "Askuala" meaning school/university) is a comprehensive, interactive web platform designed to solve this information asymmetry. It serves as a centralized hub providing deep, actionable insights into every Ethiopian university. From detailed program breakdowns, campus galleries, and climate data, to geospatial proximity and highlighting the new tier of Autonomous Universities, Finask empowers students to make data-driven educational choices rather than relying on unreliable word-of-mouth.

2. Statement of the Problem

Currently, the university selection process for Ethiopian Grade 12 students is heavily flawed due to the following critical issues:

- Severe Information Fragmentation: There is no centralized database detailing the specific programs, facilities, and campus environments of the 40+ public universities in Ethiopia.
- Blind Decision Making: Students frequently select universities based on rumors or peer pressure. They lack vital context regarding a university's geographical location, local climate, cultural environment, or distance from their hometown.
- Lack of Program Clarity: Freshman students often join specific departments without understanding the required skills, the actual courses they will take, or the subsequent career paths.
- Absence of a Support Community: Incoming students have no platform to connect with current university students to ask questions, read authentic reviews, or seek mentorship, leading to high anxiety and mismatched expectations.

3. Objectives of the Project

General Objective

To design and develop a comprehensive, interactive, and community-driven web platform that centralizes Ethiopian university data and utilizes recommendation algorithms to guide Grade 12 students in their higher education selection process.

Specific Objectives

1. Develop a Comprehensive University Database: Create a highly relational database architecture to store detailed profiles for universities, campuses, specific degree programs, and host cities (including climate, cost of living, and cultural context).
2. Implement a Smart Recommendation Engine: Build an algorithm that recommends universities and programs based on a student's specific inputs, such as their ESSLCE score predictions, preferred climate, geographical proximity to their home region, and personal career interests.
3. Integrate Geospatial & Map Features: Utilize mapping APIs (e.g., Mapbox or Google Maps) to visually display university locations, calculate distances from the capital (Addis Ababa) or the student's hometown, and provide visual context of the surrounding city.
4. Build the "Finask Community" Architecture: Develop social features that allow authenticated users to leave university/program reviews, participate in Q&A forums, and create student profiles to foster peer-to-peer connection.
5. Highlight Key Institutional Changes: Create dedicated sections to explain and highlight the newly transitioning "Autonomous Universities" (like AAU) and feature inspiring alumni ("Great Figures") to motivate incoming freshmen.

4. Significance of the Project

The Finask project has profound socio-economic and technical significance:

- Socio-Educational Impact: By democratizing access to university information, Finask reduces the stress and anxiety associated with the MoE placement process. It helps lower university dropout rates by ensuring students are placed in environments and programs that actually align with their expectations and capabilities.
- Alignment with National Educational Reforms: As Ethiopia transitions several universities to autonomous status, Finask acts as the perfect bridge to educate the public and students on what these new designations mean for their education.
- Technical Complexity (Academic Value): For the engineering team, this project demonstrates the ability to manage complex, relational data structures. It

showcases skills in building recommendation algorithms, integrating third-party mapping APIs, developing secure community forums, and creating highly interactive, user-centric frontend designs.

Concept Note: AI-Powered Project Scaffolder

1. Introduction

The initial phase of software development involves repetitive, time-consuming tasks like configuring build tools, wiring databases, and structuring directories. Standard CLI tools (e.g., `vite`) are often rigid, while Large Language Models (LLMs) frequently generate hallucinated, broken configurations when asked to build projects from scratch.

Dev-Genie is an AI-powered web platform designed to bridge this gap. By utilizing a hybrid approach that combines pre-tested foundational templates with an AI-driven orchestration layer, Dev-Genie generates complete, customizable, and error-free boilerplate code. It provides downloadable `.zip` files tailored to user requirements, automatically enforcing industry-standard architectural patterns (e.g., MVC, Domain-Driven Design) right out of the box.

2. Statement of the Problem

The industry faces a persistent bottleneck during project initialization, characterized by:

- **Setup Friction:** Developers waste hours configuring environments and rewriting identical foundational code (e.g., JWT authentication) for new projects.
- **Poor Architectural Practices:** Without guidance, junior developers often start with poor directory structures, leading to tightly coupled, unmaintainable "spaghetti code."
- **Inadequate AI Tools:** General AI models struggle to generate compilable, multi-file software projects reliably due to context limitations and hallucinated dependencies.
- **Rigid Scaffolding:** Existing generators do not adapt to user skill levels, lacking conversational guidance for beginners and granular architectural control for advanced engineers.

3. Objectives of the Project

General Objective

To develop an AI-augmented web application that dramatically reduces "Time-to-First-Feature" by generating customized, error-free, and architecturally sound boilerplate codebases for full-stack software projects.

Specific Objectives

1. **Develop a Dual-Flow UI:** Create a **"Junior Flow"** (conversational AI interface) and an **"Advanced Flow"** (granular visual dashboard for selecting specific tech stacks and architectures).

2. **Implement an Interactive Preview:** Build a dynamic frontend component that visually displays and allows customization of the proposed folder tree before generation.
3. **Engineer a Hybrid Engine:** Develop a backend system that merges hardcoded, industry-standard folder blueprints with AI-generated, context-specific files (e.g., database schemas).
4. **Automate Configurations:** Dynamically inject dependencies into package managers (`package.json`) and generate populated environment variable files (`.env.example`).
5. **Generate Context-Aware Documentation:** Utilize the LLM to write a custom `README.md` detailing exact setup instructions specific to the downloaded stack.

4. Significance of the Project

- **For the Developer Community:** Dev-Genie accelerates the development lifecycle, eliminates setup friction, and enforces scalable folder structures from day one, acting as an automated "Tech Lead."
- **Educational Value:** The platform serves as an interactive learning tool, bridging theoretical software design and practical implementation by exposing users to various architectural paradigms.
- **For the Engineering Team:** Building this platform demonstrates the team's mastery in three complex areas:
    1. *Advanced Prompt Engineering:* Ensuring reliable JSON/code output from LLMs.
    2. *Complex Backend Systems:* Handling dynamic file I/O, templating engines, and server-side stream-zipping.
    3. *Frontend State Management:* Building an interactive UI to manage deep nested states for the visual architecture builder.

Concept Note: Tankwa Tours

1. Introduction

The tourism and tour-operations industry often relies on scattered tools: separate websites for discovery and booking, manual coordination between tour operators and head office, and limited real-time communication. Customers face friction when searching, comparing, and booking tours, while operators and staff spend time on approvals, payments, and support.

Tankwa Tours is a full-stack, multi-tenant web platform that unifies tour discovery, booking, and operations in one ecosystem. It consists of a Django REST API backend and four front-end applications: a customer-facing site, an admin dashboard (admin/ERP), a tour operator's portal, and a help center. The platform uses role-based access (customers, tour operators, agents, sales, finance, admin), a structured booking and payment workflow, and real-time notifications via WebSockets so that travelers, operators, and staff can interact through a single, coherent system.

2. Statement of the Problem

The project addresses the following issues:

Fragmented experience: Tour discovery, booking, and post-booking support are often split across multiple sites or manual processes, leading to a poor customer experience and extra operational overhead.

Inefficient operations: Tour operators, sales, and finance teams lack a unified workflow for creating tours, approving content, processing payments, and handling cancellations/refunds, which increases delays and errors.

Limited visibility and trust: Travelers lack a single place to find tours, read reviews, get inspiration from articles and “things to do,” and access FAQs, which affects trust and conversion.

Delayed communication: Without real-time updates, stakeholders (e.g. operators, admin, customers) are not promptly informed about bookings, approvals, or cancellations, which slows response times and creates confusion.

3. Objectives of the Project

General Objective

To design and implement an integrated tour booking and operations platform (Tankwa Tours) that reduces friction for travelers and streamlines workflows for tour operators and staff through a single API, multiple role-specific frontends, and real-time notifications.

Specific Objectives

Unified API: Build a Django REST API that supports authentication (JWT with HttpOnly cookies, optional OAuth), role-based permissions, and modules for users, tours, bookings, reviews, things to do, articles, notifications, contact reports, and help center content.
Multi-tenant frontends: Deliver four client applications—customer site, admin dashboard, operators portal, and help center—each consuming the same API and tailored to different user roles and use cases.
Tour and booking lifecycle: Implement end-to-end flows: tour creation and approval, schedules and participant categories, itineraries (with experiences, accommodations, images), availability check, cart, checkout, booking creation, traveler management, payment processing, and cancellation/refund handling with role-based rules.

Real-time notifications: Integrate Django Channels and WebSockets (with Redis as channel layer in production) so that users receive live updates for bookings, approvals, and other critical events.
Customer experience: Provide tour discovery, search/filter, wishlist, cart, multi-step checkout, user accounts, bookings history, reviews, travel articles, “things to do,” interactive maps.
Operational tooling: Provide admin and operators with dashboards for user management, tour approval/activation, booking and payment management, cancellation/refund decisions, operator financial summaries, and help center content management.
4. Significance of the Project
For the tourism sector: Tankwa Tours demonstrates how a single platform can connect travelers, tour operators, and internal teams, improving discoverability, conversion, and operational efficiency.

Backend design: RESTful API design, JWT and OAuth, role-based permission models, and integration of async features (Channels, WebSockets) with a Django monolith.
Multi-frontend architecture: One API serving several SPAs (React/Vite) with different UIs and permissions, and CORS/security configured for multiple origins.
Business logic: Complex workflows (approval, payments, refunds, cancellation requests) and consistent permission checks across views and serializers.